Communication Preserving SFE from Obfuscation

Omkant Pandey Christopher Smith Yuhao Tang

Stony Brook University

TCC 2026

eprint 2026/2214

Secure Function Evaluation

Communication Preserving SFE from Obfuscation

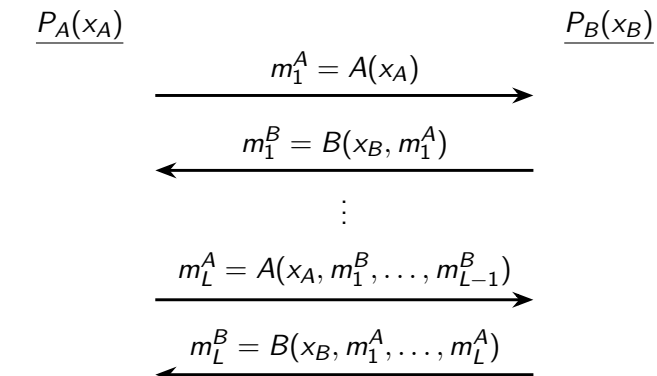
Omkant Pandey Christopher Smith Yuhao Tang

Stony Brook University

TCC 2026

eprint 2026/2214

Basic Communication Model



- Communication complexity cc is minimum number of bits we need to exchange in order to compute $(y_A, y_B) = f(x_A, x_B)$ **insecurely**.
- (A, B) are next-message Turing machines.

Motivation

- Sending input is not always communication-optimal!
- Functions like "greater-than" and "k-th smallest element" have insecure protocols communication sublinear in the input size $|x_A| + |x_B|$.
- How close can communication complexity $\widetilde{\mathrm{cc}}$ of **secure** function evaluation be to insecure complexity cc?

Prior Work

Common strategy: develop insecure-to-secure protocol compiler and analyze *rate* $\mathsf{cc}/\widetilde{\mathsf{cc}}$.

Prior Work

Common strategy: develop insecure-to-secure protocol compiler and analyze *rate* $\mathsf{cc}/\widetilde{\mathsf{cc}}$.

- Naor and Nissim [NN01] give compiler with $\widetilde{\mathsf{cc}} \leq \mathsf{cc} \cdot \mathsf{poly}(\lambda)$

Prior Work

Common strategy: develop insecure-to-secure protocol compiler and analyze *rate* $\mathsf{cc}/\widetilde{\mathsf{cc}}$.

- Naor and Nissim [NN01] give compiler with $\widetilde{\mathsf{cc}} \leq \mathsf{cc} \cdot \mathrm{poly}(\lambda)$
- FHE [Gen09] gives a trivial compiler matching $\widetilde{\mathsf{cc}} \leq \mathsf{cc} \cdot \mathrm{poly}(\lambda)$.

Prior Work

Common strategy: develop insecure-to-secure protocol compiler and analyze *rate* $\mathsf{cc}/\widetilde{\mathsf{cc}}$.

- Naor and Nissim [NN01] give compiler with $\widetilde{\mathsf{cc}} \leq \mathsf{cc} \cdot \mathsf{poly}(\lambda)$
- FHE [Gen09] gives a trivial compiler matching $\widetilde{\mathsf{cc}} \leq \mathsf{cc} \cdot \mathsf{poly}(\lambda)$.
- Hubáček and Wichs [HW15] show $\widetilde{\mathsf{cc}} \geq \ell_f$ where ℓ_f is output length of f.[1]

[1] In light of [HW15] result, we actually define rate as $(\max(\mathsf{cc}, \ell_f) + \mathsf{poly}(\lambda))/\widetilde{\mathsf{cc}}$

Prior Work

Common strategy: develop insecure-to-secure protocol compiler and analyze *rate* $\mathsf{cc}/\widetilde{\mathsf{cc}}$.

- Naor and Nissim [NN01] give compiler with $\widetilde{\mathsf{cc}} \leq \mathsf{cc} \cdot \mathsf{poly}(\lambda)$
- FHE [Gen09] gives a trivial compiler matching $\widetilde{\mathsf{cc}} \leq \mathsf{cc} \cdot \mathsf{poly}(\lambda)$.
- Hubáček and Wichs [HW15] show $\widetilde{\mathsf{cc}} \geq \ell_f$ where ℓ_f is output length of f.[1]
- Compiler of Döttling, Goyal, Malavolta and Raizes [DGMR22] gives

$$\widetilde{\mathsf{cc}} \leq \mathsf{cc} + o(cc) + (L + \ell_f) \cdot \mathsf{poly}(\lambda)$$

Rate-1 for $L = \ell_f = O(1)$, but rate-0 in general (what if $L \approx \mathsf{cc}$?).

[1] In light of [HW15] result, we actually define rate as $(\max(\mathsf{cc}, \ell_f) + \mathsf{poly}(\lambda))/\widetilde{\mathsf{cc}}$

Results

Q: What is the highest rate compiler we can achieve? Under what assumptions?

Results

Q: What is the highest rate compiler we can achieve? Under what assumptions?

- **Main Result:** Constant rate[2] in plain model assuming subexponentially secure input-succinct i$\mathcal{O}$ for Turing machines and standard assumptions.

[2] *i.e.*, $\widetilde{\mathsf{cc}} = O(\mathsf{cc}) + \mathsf{poly}(\lambda)$

Results

Q: What is the highest rate compiler we can achieve? Under what assumptions?

- **Main Result:** Constant rate[2] in plain model assuming subexponentially secure input-succinct i$\mathcal{O}$ for Turing machines and standard assumptions.
- **Corollary:** Constant rate in $\{$CRS model | ROM model$\}$ assuming subexponentially secure i$\mathcal{O}$ for circuits and standard assumptions.

[2] *i.e.*, $\widetilde{\mathsf{cc}} = O(\mathsf{cc}) + \mathsf{poly}(\lambda)$

Input-succinct i$\mathcal{O}$ for Turing machines [JJMP25]

- Syntax: $\mathsf{ob}M \leftarrow \mathsf{i}\mathcal{O}\mathsf{M}(1^\lambda, M)$
- Correctness: $\mathsf{ob}M(x) = M(x)$ for all x
- Security: $\mathsf{ob}M_0 \overset{\mathsf{c}}{\approx} \mathsf{ob}M_1$ for all functionally equivalent M_0, M_1 of equal length.
- **Input-succinctness:** $|\mathsf{ob}M| = \mathsf{poly}(\lambda, |M|)$

Input-succinct i$\mathcal{O}$ for Turing machines [JJMP25]

- Syntax: $\mathsf{ob}M \leftarrow \mathsf{i}\mathcal{O}\mathsf{M}(1^\lambda, M)$
- Correctness: $\mathsf{ob}M(x) = M(x)$ for all x
- Security: $\mathsf{ob}M_0 \overset{\mathsf{c}}{\approx} \mathsf{ob}M_1$ for all functionally equivalent M_0, M_1 of equal length.
- **Input-succinctness:** $|\mathsf{ob}M| = \mathsf{poly}(\lambda, |M|)$

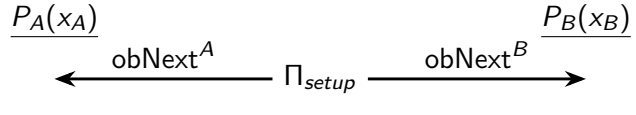
- Known from non-falsifiable extractability-type assumptions like di$\mathcal{O}$ / pc-di$\mathcal{O}$.
- Known from falsifiable assumptions in CRS model, or in plain model for PV-equivalent machines.

Rough Starting Point

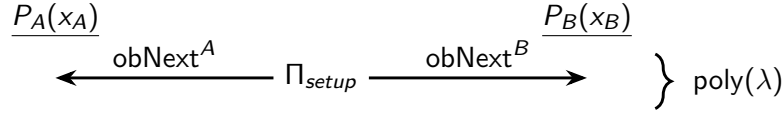
$\underline{P_A(x_A)}$

$\underline{P_B(x_B)}$

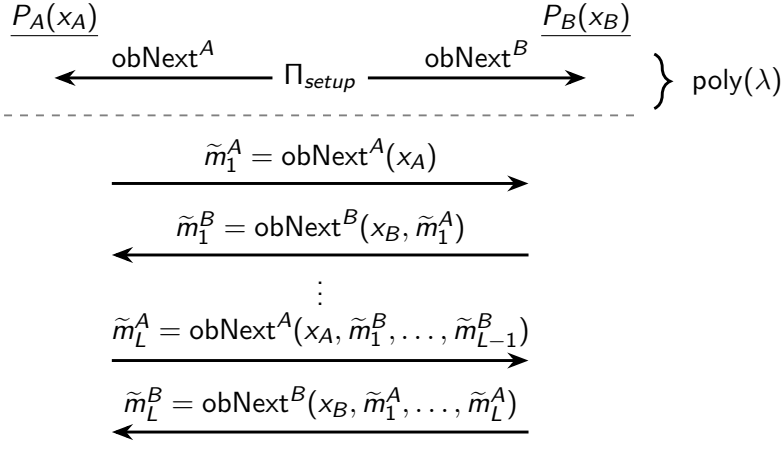
Rough Starting Point



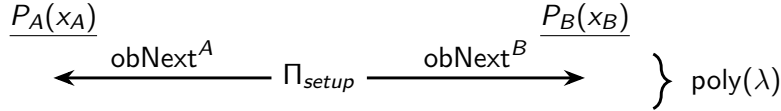
Rough Starting Point



Rough Starting Point

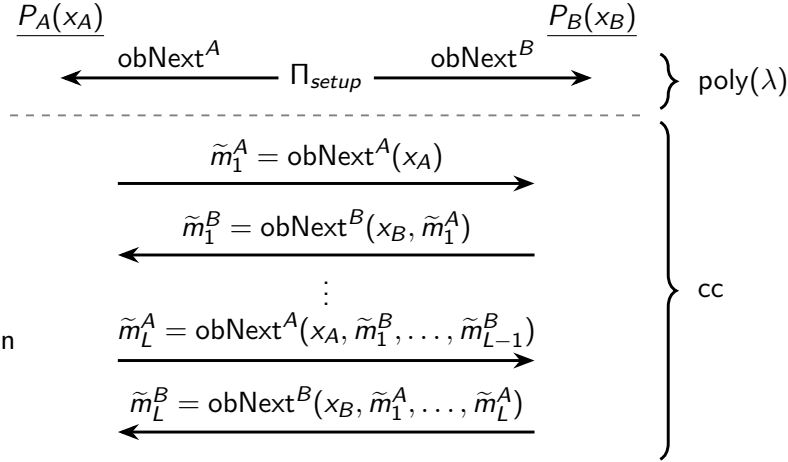


Rough Starting Point



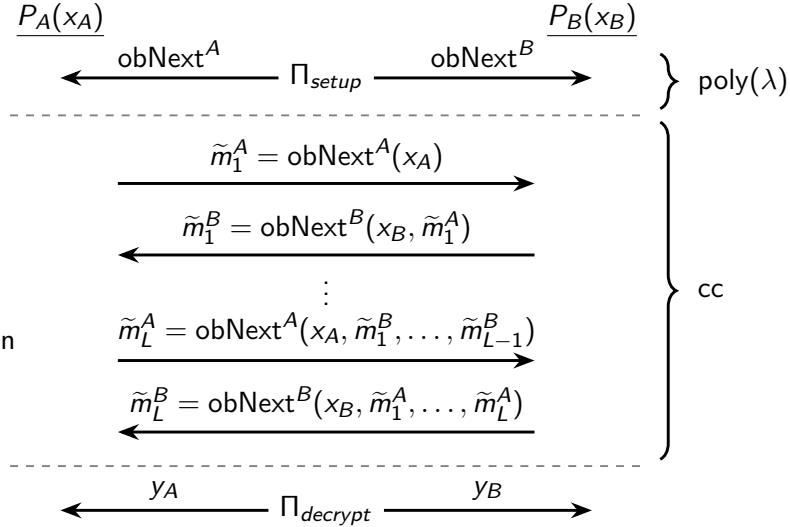
- $\widetilde{m}_i^A = m_i^A \oplus F_k(\mathrm{p}_i^A)$
- $\widetilde{m}_i^B = m_i^B \oplus F_k(\mathrm{p}_i^B)$
- p_i^A and p_i^B depend on transcript thus far.

Rough Starting Point



- $\widetilde{m}_i^A = m_i^A \oplus F_k(\mathrm{p}_i^A)$
- $\widetilde{m}_i^B = m_i^B \oplus F_k(\mathrm{p}_i^B)$
- p_i^A and p_i^B depend on transcript thus far.

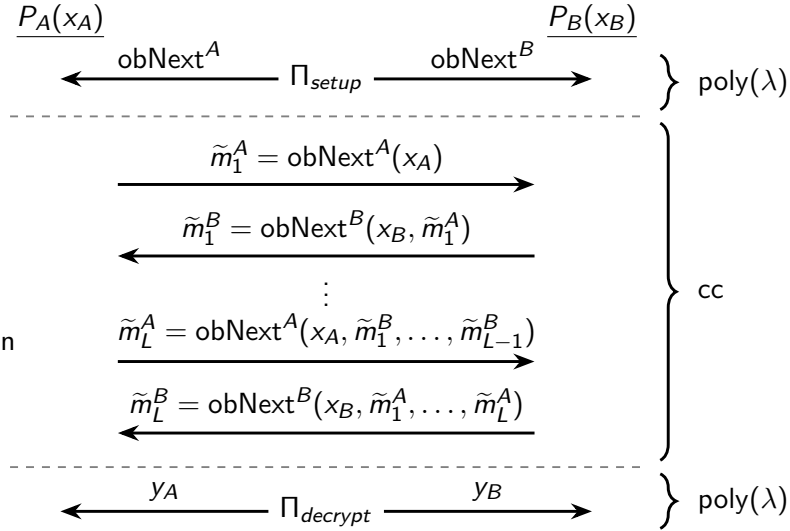
Rough Starting Point



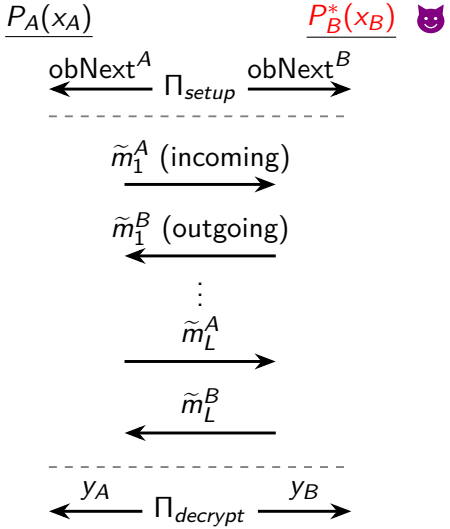
- $\widetilde{m}_i^A = m_i^A \oplus F_k(\mathrm{p}_i^A)$
- $\widetilde{m}_i^B = m_i^B \oplus F_k(\mathrm{p}_i^B)$
- p_i^A and p_i^B depend on transcript thus far.
- assuming short outputs...

Rough Starting Point

- $\widetilde{m}_i^A = m_i^A \oplus F_k(\mathrm{p}_i^A)$
- $\widetilde{m}_i^B = m_i^B \oplus F_k(\mathrm{p}_i^B)$
- p_i^A and p_i^B depend on transcript thus far.
- assuming short outputs...

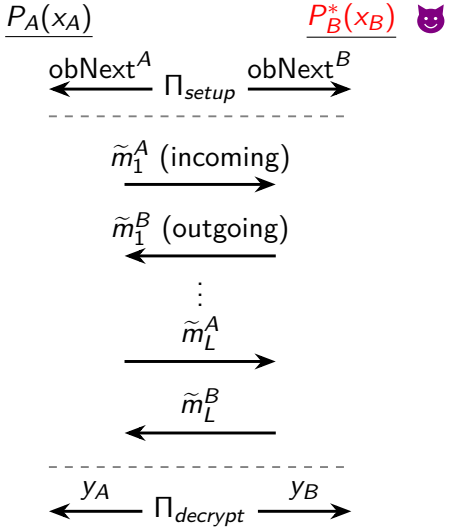


Towards a Security Proof...



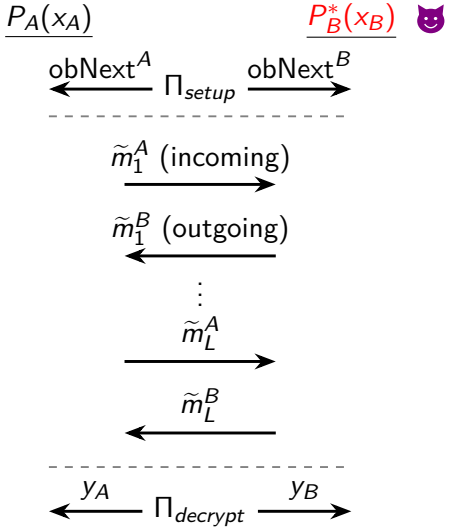
- Want to change $\left\{\widetilde{m}_i^A\right\}$ to (pseudo-)random, independent of x_A, in simulation for adversarial P_B^*.

Towards a Security Proof...



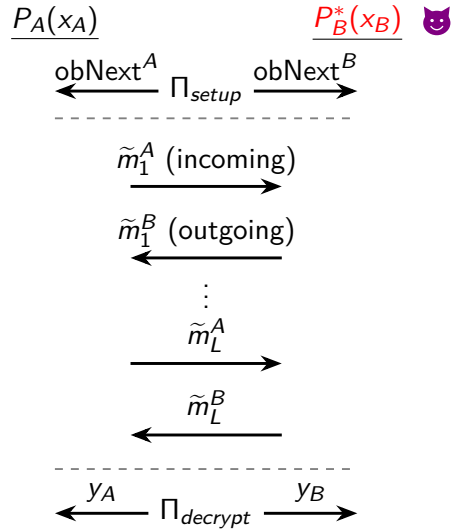
- Want to change $\left\{\widetilde{m}_i^A\right\}$ to (pseudo-)random, independent of x_A, in simulation for adversarial P_B^*.
- But $\left\{\widetilde{m}_i^A\right\}$ are correlated with key k in Next^B.

Towards a Security Proof...



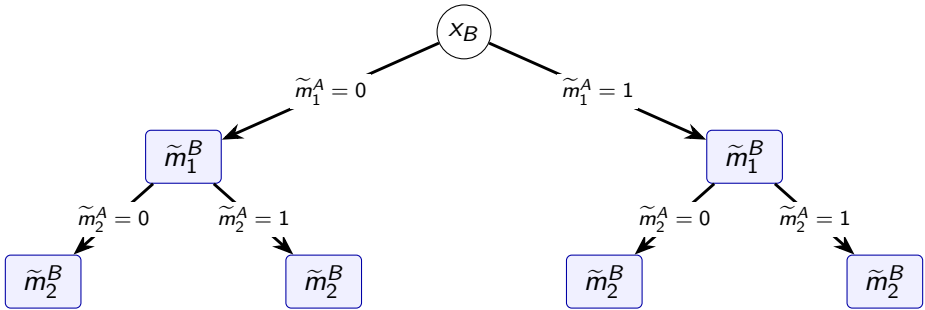
- Want to change $\left\{\widetilde{m}_i^A\right\}$ to (pseudo-)random, independent of x_A, in simulation for adversarial P_B^*.
- But $\left\{\widetilde{m}_i^A\right\}$ are correlated with key k in Next^B.
- Goal: Remove k from Next^B.

Towards a Security Proof...



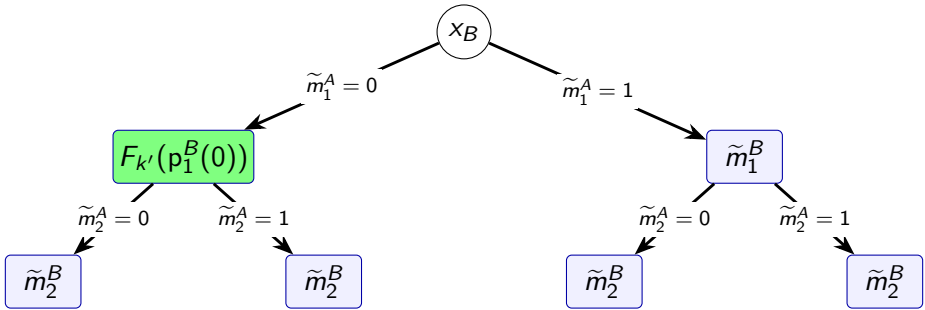
- Want to change $\{\widetilde{m}_i^A\}$ to (pseudo-)random, independent of x_A, in simulation for adversarial P_B^*.
- But $\{\widetilde{m}_i^A\}$ are correlated with key k in Next^B.
- Goal: Remove k from Next^B.
- Idea: can modify Next^B at individual point to output $\widetilde{m}_i^B = F_{k'}(\mathrm{p}_i^B)$, for new "dummy" key k$'$.

The Next$^B(x_B, \cdot)$ Tree



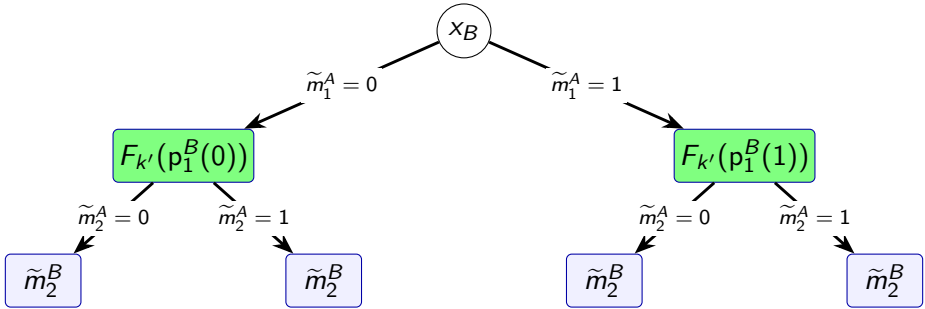
- Assume Next^B checks first input is equal to hardcoded x_B (will remove later).

The $\mathsf{Next}^B(x_B, \cdot)$ Tree



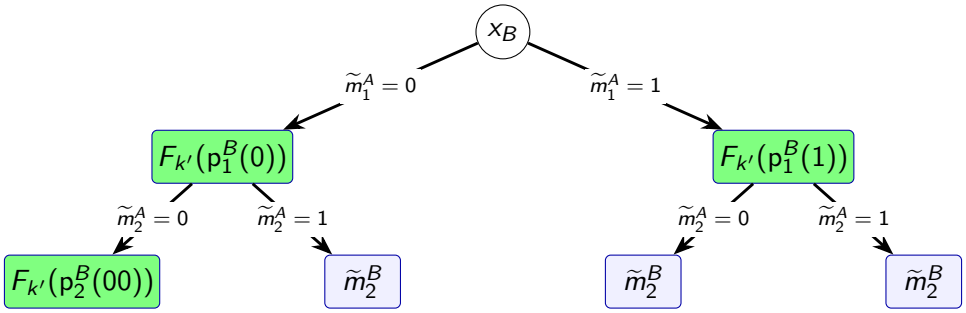
- Assume Next^B checks first input is equal to hardcoded x_B (will remove later).
- Can change **a single** $\widetilde{m}_i^B$ to dummy with $O(1)$ hybrids...

The Next$^B(x_B, \cdot)$ Tree



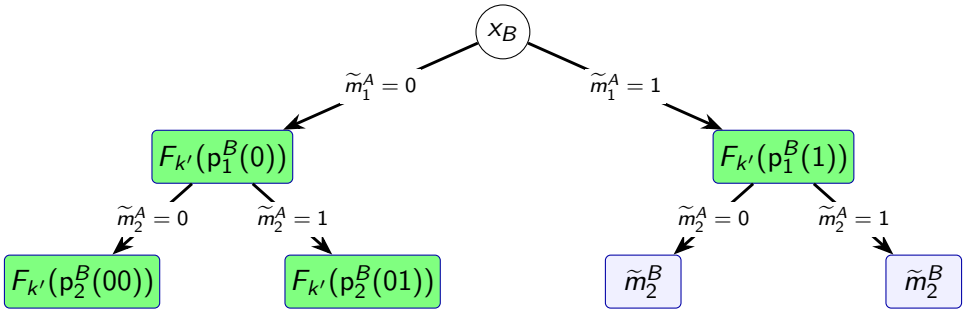
- Assume Next^B checks first input is equal to hardcoded x_B (will remove later).
- Can change **a single** $\widetilde{m}_i^B$ to dummy with $O(1)$ hybrids...

The Next$^B(x_B, \cdot)$ Tree



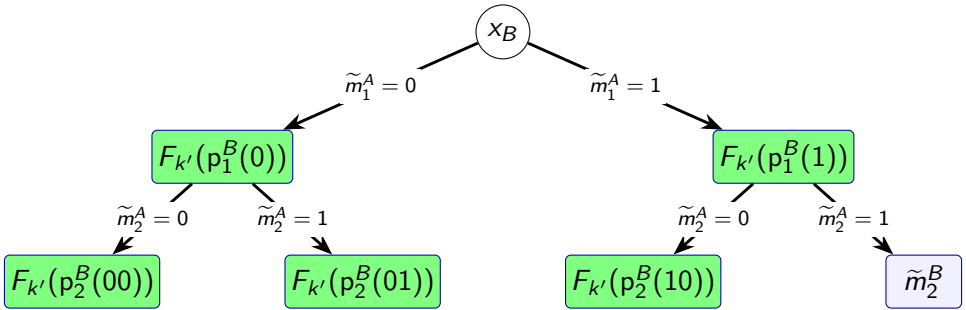
- Assume Next^B checks first input is equal to hardcoded x_B (will remove later).
- Can change **a single** $\widetilde{m}_i^B$ to dummy with $O(1)$ hybrids...

The $\mathsf{Next}^B(x_B, \cdot)$ Tree



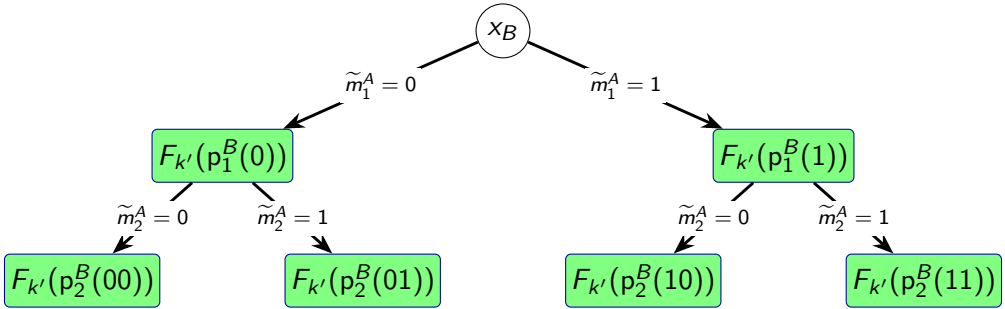
- Assume Next^B checks first input is equal to hardcoded x_B (will remove later).
- Can change **a single** $\widetilde{m}_i^B$ to dummy with $O(1)$ hybrids...

The $\mathsf{Next}^B(x_B, \cdot)$ Tree



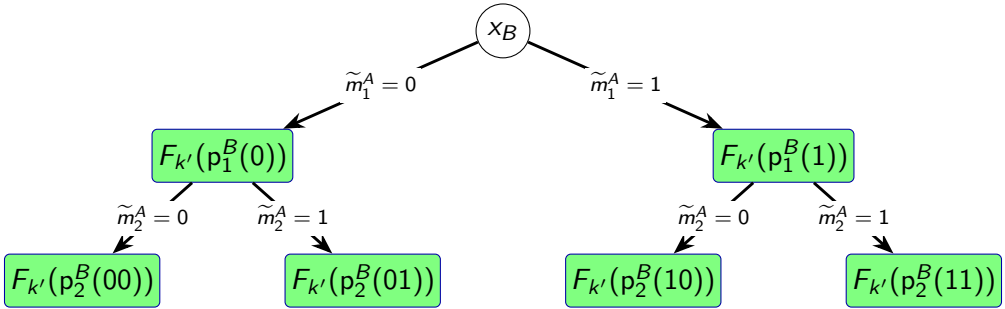
- Assume Next^B checks first input is equal to hardcoded x_B (will remove later).
- Can change **a single** $\widetilde{m}_i^B$ to dummy with $O(1)$ hybrids...

The $\mathsf{Next}^B(x_B, \cdot)$ Tree



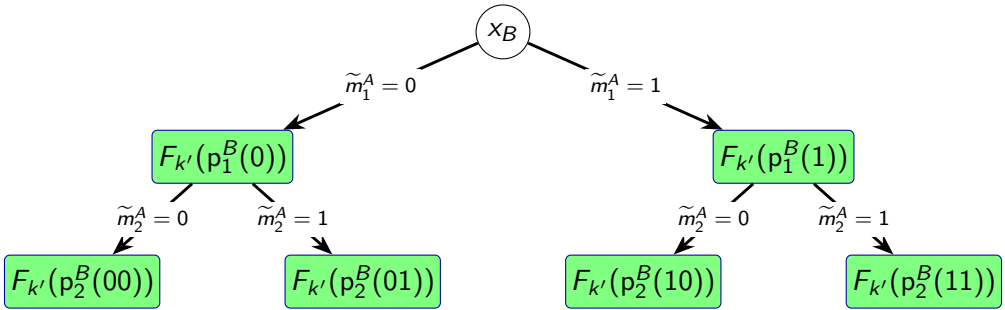
- Assume Next^B checks first input is equal to hardcoded x_B (will remove later).
- Can change **a single** $\widetilde{m}_i^B$ to dummy with $O(1)$ hybrids...

The $\mathsf{Next}^B(x_B, \cdot)$ Tree



- Assume Next^B checks first input is equal to hardcoded x_B (will remove later).
- Can change **a single** $\widetilde{m}_i^B$ to dummy with $O(1)$ hybrids...

The Next$^B(x_B, \cdot)$ Tree



- Assume Next^B checks first input is equal to hardcoded x_B (will remove later).
- Can change **a single** $\widetilde{m}_i^B$ to dummy with $O(1)$ hybrids...
- Issue: too many hybrids!

Idea: Prune the Tree

- Divide protocol into segments. Exchange signatures after each segment, on messages sent in that segment.

Idea: Prune the Tree

- Divide protocol into segments. Exchange signatures after each segment, on messages sent in that segment.
- $\mathsf{Next}^A, \mathsf{Next}^B$ verify signatures, and output $\perp$ on failure.

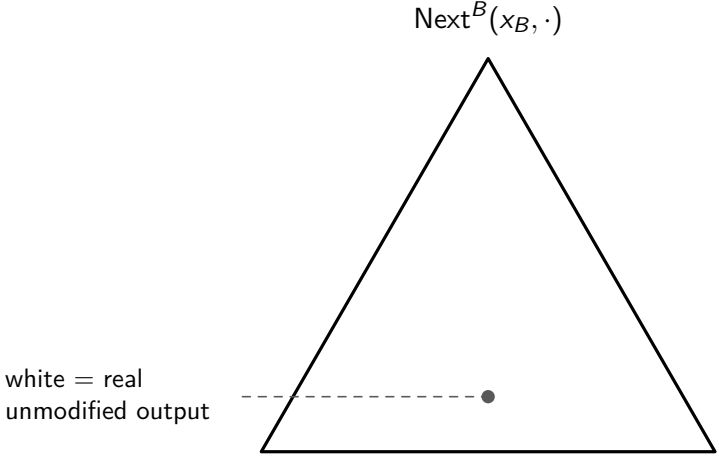
Idea: Prune the Tree

- Divide protocol into segments. Exchange signatures after each segment, on messages sent in that segment.
- $\mathsf{Next}^A, \mathsf{Next}^B$ verify signatures, and output $\perp$ on failure.
- Our "dual-mode programmable one-time signature" scheme DPSig allows hybrid to program verification key to **only accept signatures on a unique segment**.

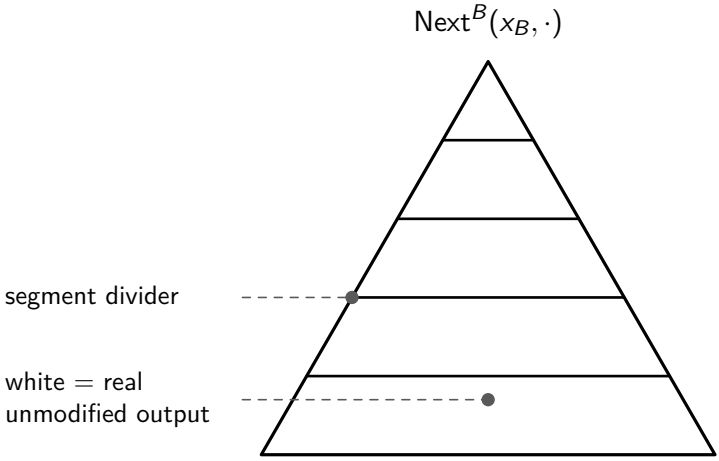
Idea: Prune the Tree

- Divide protocol into segments. Exchange signatures after each segment, on messages sent in that segment.
- $\mathsf{Next}^A, \mathsf{Next}^B$ verify signatures, and output $\perp$ on failure.
- Our "dual-mode programmable one-time signature" scheme DPSig allows hybrid to program verification key to **only accept signatures on a unique segment**.
- Upshot: complement of subtree rooted at this segment becomes $\perp$...

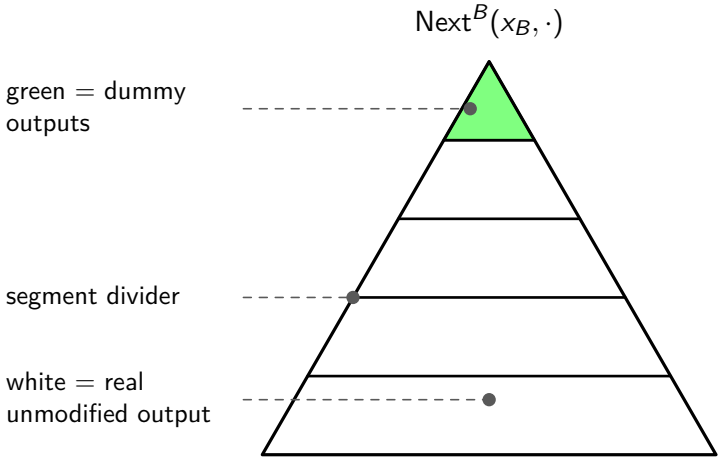
Pruning the Tree



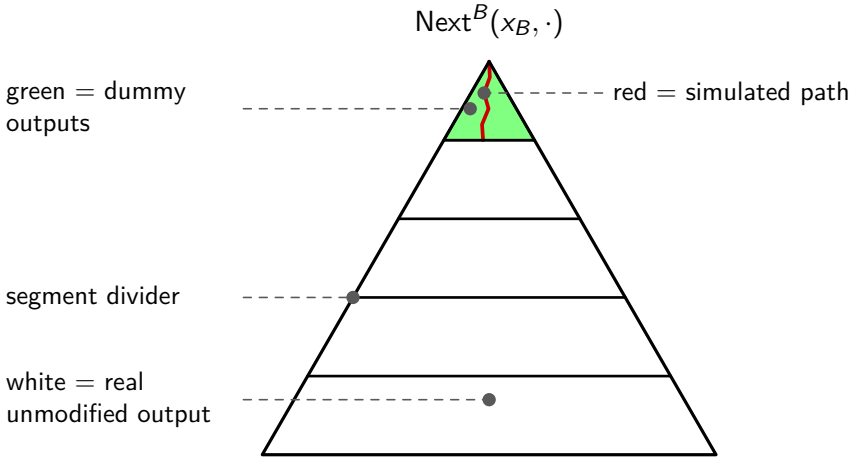
Pruning the Tree



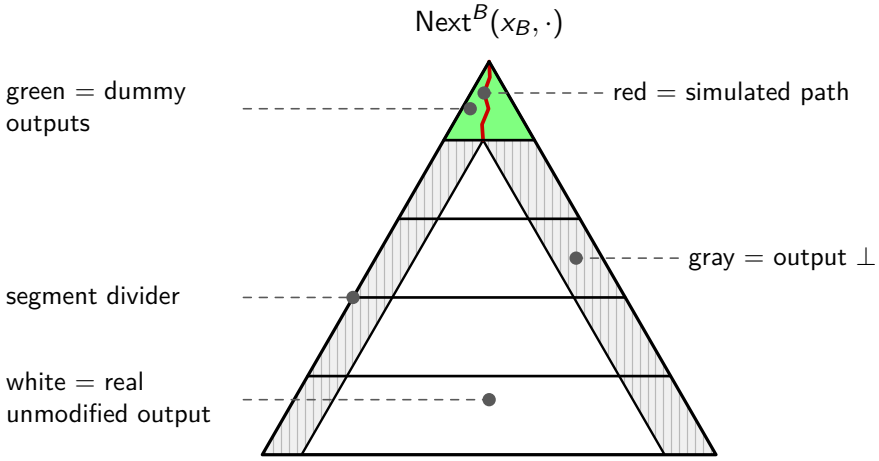
Pruning the Tree



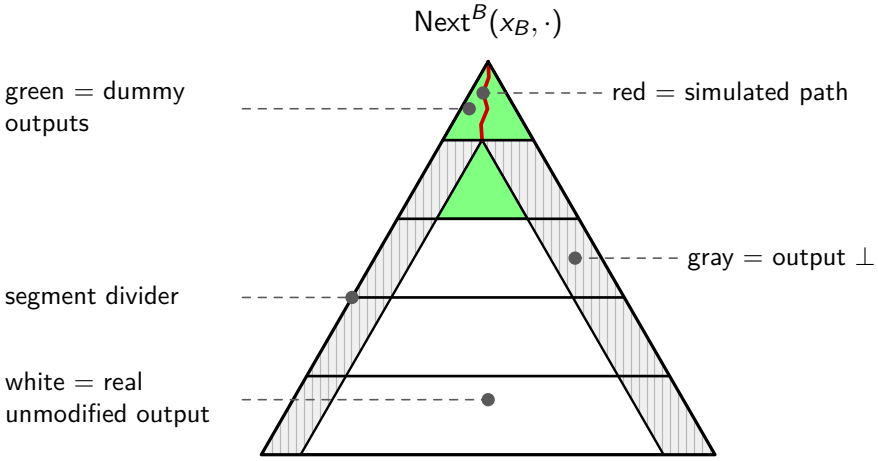
Pruning the Tree



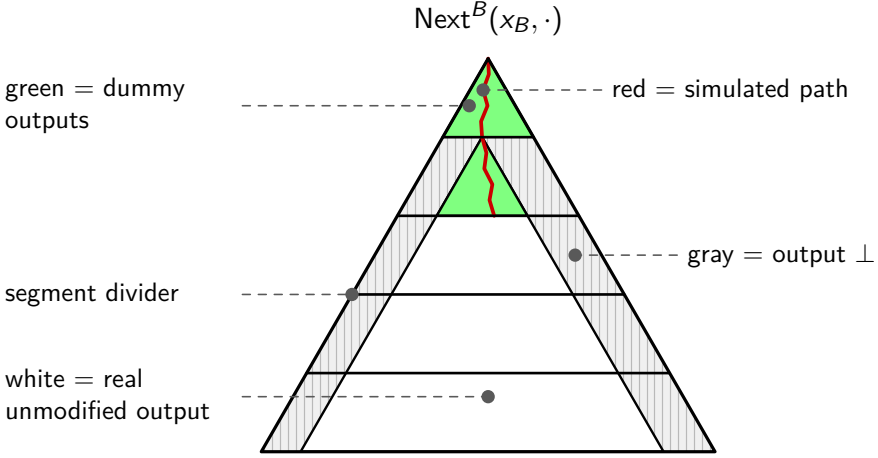
Pruning the Tree



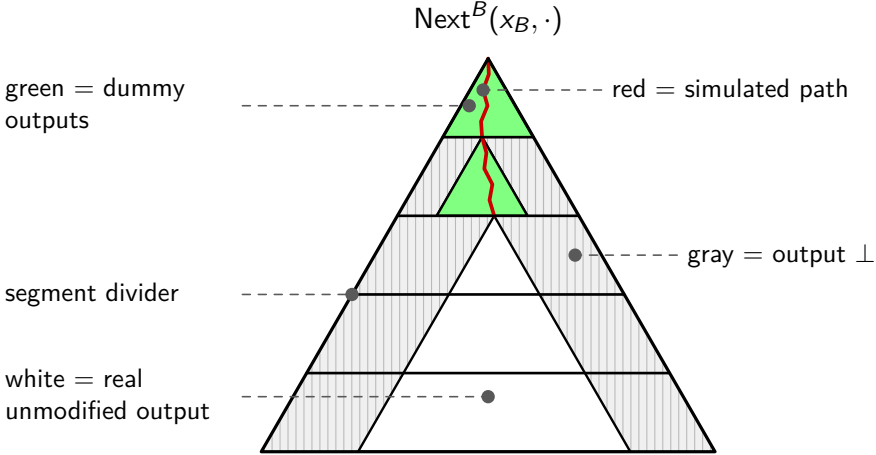
Pruning the Tree



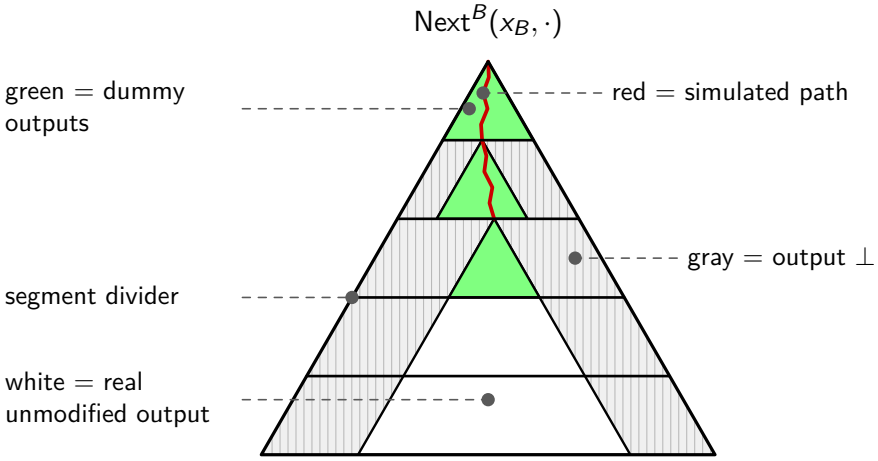
Pruning the Tree



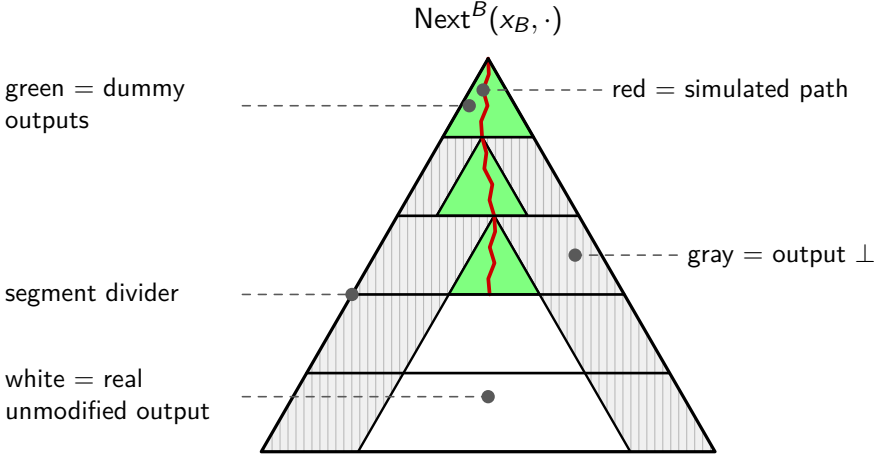
Pruning the Tree



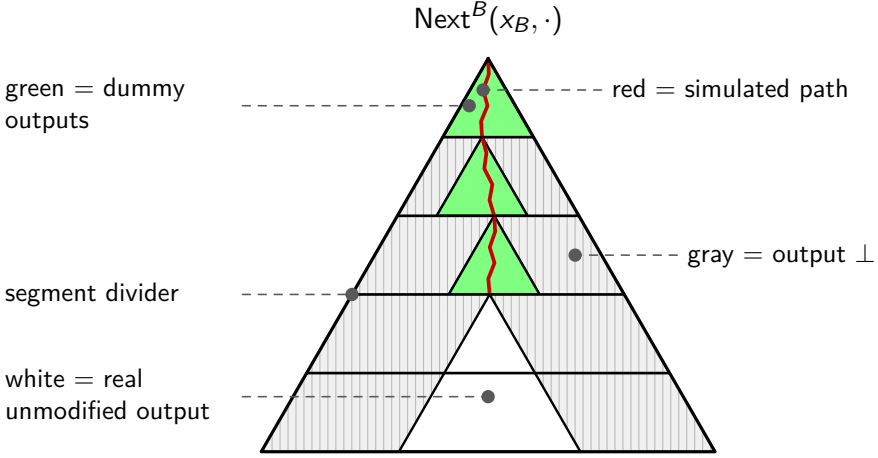
Pruning the Tree



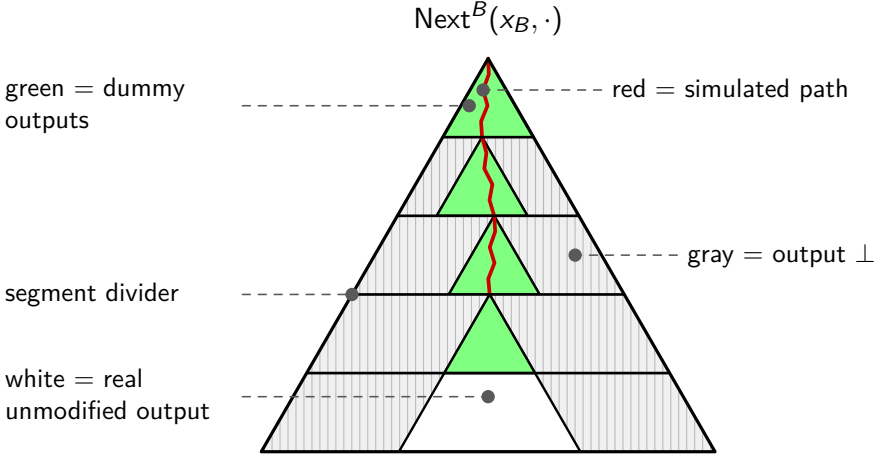
Pruning the Tree



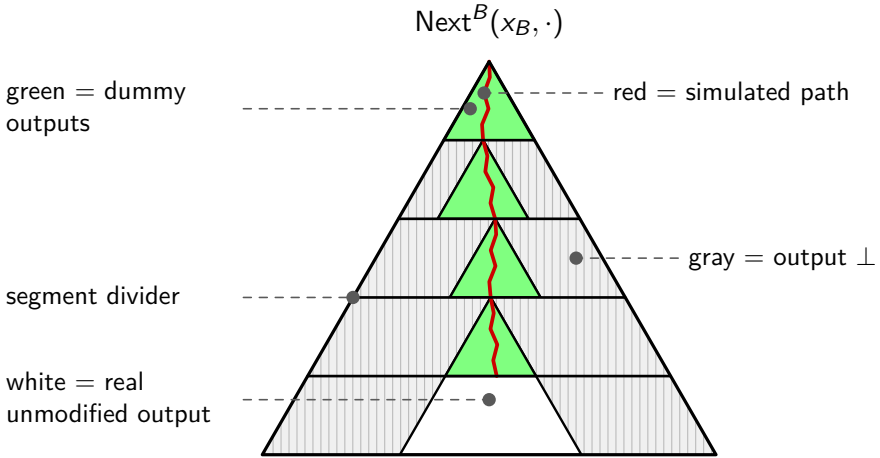
Pruning the Tree



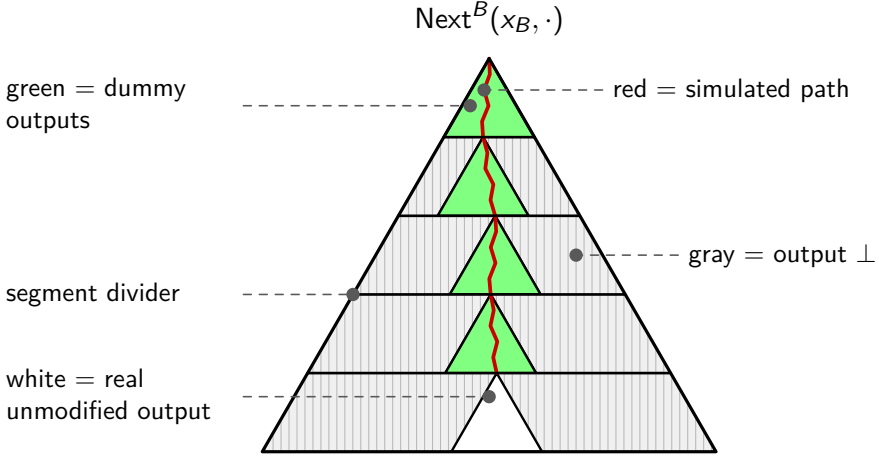
Pruning the Tree



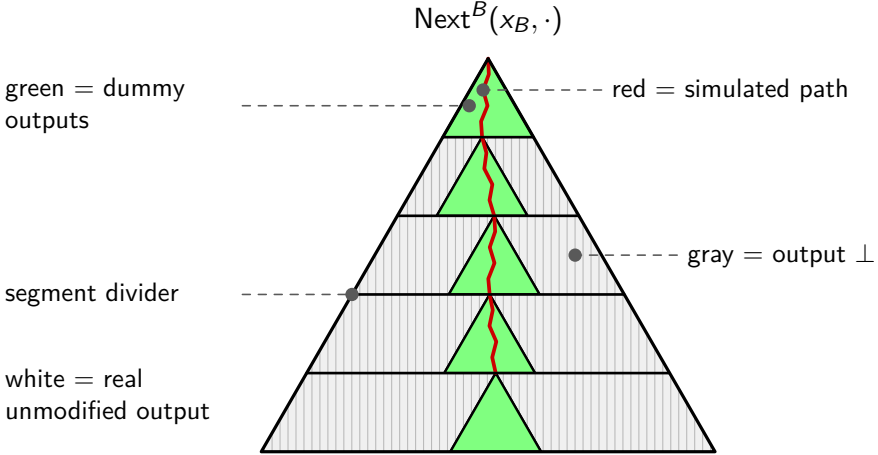
Pruning the Tree



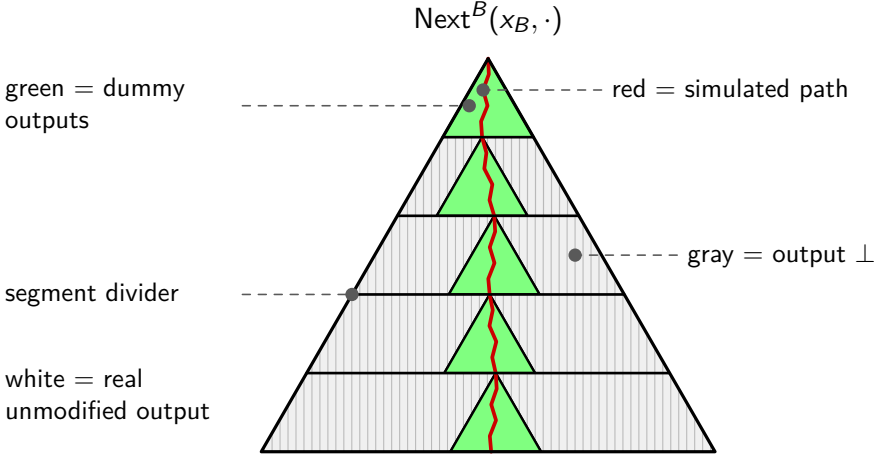
Pruning the Tree



Pruning the Tree



Pruning the Tree



Function Binding Hash

- Want to replace hardcoded x_B with hash h_B to improve rate.

Function Binding Hash

- Want to replace hardcoded x_B with hash h_B to improve rate.
- May exist bad collisions $x \neq x_B$ where Next^B trees differ $\Rightarrow$ can't use i$\mathcal{O}$.

Function Binding Hash

- Want to replace hardcoded x_B with hash h_B to improve rate.
- May exist bad collisions $x \neq x_B$ where $\mathsf{Next}^{\mathcal{B}}$ trees differ $\Rightarrow$ can't use i$\mathcal{O}$.
- To avoid di$\mathcal{O}$, suffices for hash function H to be statistically "next-message binding" for the insecure protocol. Roughly: "hashes match" $\implies$ "$\mathcal{B}$ outputs match".

Function Binding Hash

- Want to replace hardcoded x_B with hash h_B to improve rate.
- May exist bad collisions $x \neq x_B$ where Next^B trees differ $\Rightarrow$ can't use i$\mathcal{O}$.
- To avoid di$\mathcal{O}$, suffices for hash function H to be statistically "next-message binding" for the insecure protocol. Roughly: "hashes match" $\implies$ "B outputs match".
- We build an FHE-based "function-binding hash" (for TMs) to achieve this.

Function Binding Hash

- Want to replace hardcoded x_B with hash h_B to improve rate.
- May exist bad collisions $x \neq x_B$ where Next^B trees differ $\Rightarrow$ can't use i$\mathcal{O}$.
- To avoid di$\mathcal{O}$, suffices for hash function H to be statistically "next-message binding" for the insecure protocol. Roughly: "hashes match" $\implies$ "B outputs match".
- We build an FHE-based "function-binding hash" (for TMs) to achieve this.
- To hardcode h_B in obNext^B during setup, H needs a transcript to compute next message *before* m_1^A is sent...

Function Binding Hash

- Want to replace hardcoded x_B with hash h_B to improve rate.
- May exist bad collisions $x \neq x_B$ where Next^B trees differ $\Rightarrow$ can't use i$\mathcal{O}$.
- To avoid di$\mathcal{O}$, suffices for hash function H to be statistically "next-message binding" for the insecure protocol. Roughly: "hashes match" $\implies$ "B outputs match".
- We build an FHE-based "function-binding hash" (for TMs) to achieve this.
- To hardcode h_B in obNext^B during setup, H needs a transcript to compute next message *before* m_1^A is sent...
- Use CRS: $H(x, \mathsf{crs}) = h_B \implies B(x, \mathsf{Dec}_{k_{\mathsf{crs}}}(\mathsf{crs})) = B(x_B, \mathsf{Dec}_{k_{\mathsf{crs}}}(\mathsf{crs}))$

Do we really need a CRS?

Do we really need a CRS?

- Strictly speaking, no.

Do we really need a CRS?

- Strictly speaking, no.
- Parties can exchange uniform random strings before setup. No coin tossing needed!

Do we really need a CRS?

- Strictly speaking, no.
- Parties can exchange uniform random strings before setup. No coin tossing needed!
- But our proof does make heavy use of some kind of encrypted preamble / CRS, even beyond the function binding hash (see paper).

Do we really need a CRS?

- Strictly speaking, no.
- Parties can exchange uniform random strings before setup. No coin tossing needed!
- But our proof does make heavy use of some kind of encrypted preamble / CRS, even beyond the function binding hash (see paper).
- And without CRS, we need to assume input-succinct i$\mathcal{O}$ for Turing machines.

Open Questions

Can we get rate-1 under falsifiable assumptions?

Can we get constant rate in plain model without input-succinct i$\mathcal{O}$ *for Turing machines?*

Can we get constant rate under polynomial hardness?

Q&A

Thank you!

References I

[DGMR22] Nico Döttling, Vipul Goyal, Giulio Malavolta, and Justin Raizes.
Interaction-Preserving Compilers for Secure Computation.
In *13th Innovations in Theoretical Computer Science Conference (ITCS 2022)*, volume 215 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 57:1–57:18, 2022.

[Gen09] Craig Gentry.
Fully homomorphic encryption using ideal lattices.
In *Proceedings of the forty-first annual ACM symposium on Theory of computing*, pages 169–178, 2009.

References II

[HW15] Pavel Hubacek and Daniel Wichs.
On the communication complexity of secure function evaluation with long output.
In *Proceedings of the 2015 Conference on Innovations in Theoretical Computer Science*, ITCS '15, page 163–172, New York, NY, USA, 2015. Association for Computing Machinery.

[JJMP25] Abhishek Jain, Zhengzhong Jin, Surya Mathialagan, and Omer Paneth.
On succinct obfuscation via propositional proofs.
In *2025 IEEE 66th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1706–1743. IEEE, 2025.

[NN01] Moni Naor and Kobbi Nissim.
Communication preserving protocols for secure function evaluation.
In *Proceedings of the thirty-third annual ACM symposium on Theory of computing*, pages 590–599, 2001.